

Performance optimization using Entity framework core

Improving performance when using Entity Framework Core (EF Core) is crucial for scalable, responsive .NET applications. EF Core offers a wide range of optimization strategies that help minimize query latency, memory usage, and CPU overhead. Below are key best practices and advanced techniques you can apply.

1. Use AsNoTracking() for Read-Only Queries

By default, EF Core tracks all retrieved entities, which incurs overhead when you only need to read data. Use `AsNoTracking()` for read-only operations to boost speed:

Example:

```
var products = await context.Products
    .AsNoTracking()
    .ToListAsync();
```

This can improve read performance up to 40% in heavy read scenarios.

2. Limit Data Retrieval & Use Projections

Fetch only the fields you need instead of entire entity objects:

Syntax:

```
var productSummaries = await context.Products
    .Select(p => new { p.Id, p.Name, p.Price })
    .ToListAsync();
```

This minimizes both memory and data transfer, reducing query size and improving execution time.

3. Avoid the N+1 Query Problem

Lazy loading can silently trigger multiple database trips. Use eager loading (`Include`) or explicit loading instead:

```
var orders = await context.Orders
    .Include(o => o.Customer)
    .ToListAsync();
```

This ensures all required data loads in a single query rather than multiple round trips.

4. Enable DbContext Pooling

Repeated creation and disposal of DbContext instances impact performance. You can pool contexts using:

```
builder.Services.AddDbContextPool<HerbalDbContext>(options =>  
    options.UseSqlServer(configuration.GetConnectionString("DefaultConnection")));
```

This reduces garbage collection pressure and enhances throughput under high load.

5. Use Compiled Queries for Hot Paths

For frequently executed queries, compile them once to avoid repeated parsing:

```
static readonly Func<HerbalDbContext, decimal, IEnumerable<Product>>  
_expensiveCompiledQuery =  
    EF.CompileQuery((HerbalDbContext ctx, decimal price) =>  
        ctx.Products.Where(p => p.Price > price));  
var result = _expensiveCompiledQuery(context, 100);
```

Compiled queries can reduce latency by 30–50% for repeated database reads.

6. Optimize Change Tracking

When inserting or updating large entity sets, disable automatic change detection:

```
context.ChangeTracker.AutoDetectChangesEnabled = false;
```

This significantly speeds up bulk inserts or updates.

7. Use Query Batching

EF Core can execute multiple commands in a single round trip, reducing DB latency:

```
await context.SaveChangesAsync(); // Automatically batches multiple inserts/updates
```

Batching can make queries more than 3× faster under high-load write conditions.

8. Implement Caching for Reference Data

Cache rarely updated, frequently used data (like product categories or regions) using In-Memory or Distributed Caching:

```
var categories = await _cache.GetOrCreateAsync("categories", entry =>
{
    entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromHours(1);
    return context.Categories.AsNoTracking().ToListAsync();
});
```

Caching can drastically reduce database hits for static data.

9. Use Async All the Way

Always use asynchronous methods (ToListAsync(), FirstOrDefaultAsync()) to keep threads free and scale better under concurrent requests.

10. Profile and Monitor Queries

Use tools like:

- EF Core Logging (EnableSensitiveDataLogging)
- SQL Server Profiler
- Visual Studio Diagnostic Tools
- MiniProfiler or Application Insights

Regular profiling helps detect inefficient queries and unoptimized LINQ expressions.

Adopting these techniques proactively yields faster query execution (up to 50%), lower memory footprint, and higher scalability in EF Core-based.

Summary of EF Core Performance Tips

Technique	Purpose	Typical Gain
AsNoTracking	For read-only queries	30–40% faster reads
Projections	Load only needed fields	Lower memory & DB load
Eager Loading	Avoid lazy-loading overhead	Fewer DB trips
Context Pooling	Reuse DbContext	Less GC pressure
Compiled Queries	Precompile frequent queries	30–50% faster
Change Tracker Optimization	Bulk insert/update	Up to 2× faster
Caching (Memory/Redis)	Reduce DB hits	Instant reads for reference data

CAP Theorem Explanation with Example:

<https://www.geeksforgeeks.org/system-design/cap-theorem-in-system-design/>

Roadmap of React:

https://www.geeksforgeeks.org/blogs/react-roadmap/?utm_source=chatgpt.com

Questions for this assignment

What is Asp.Net Core?

What are the features of Asp.Net Core?

What are the advantages of ASP.NET Core over ASP.NET (.NET Framework)?

What is Asp.Net Core meta package?

When do you choose classic ASP.NET MVC over ASP.NET Core?

What is a web application framework, and what are its benefits?

10 Websites to Get FREE Tailwind Components

[10 Websites to Get FREE Tailwind Components - DEV Community](#)

Build CI/CD Pipeline For Azure Kubernetes

<https://www.c-sharpcorner.com/article/build-cicd-pipeline-for-azure-kubernetes/>

CI/CD Pipeline for a .NET Web API Application in Azure DevOps

[CI/CD Pipeline for a .NET Web API Application in Azure DevOps](#)

Export SQL Data to Excel Using ASP.Net

[Export SQL Data to Excel Using ASP.Net](#)